

Input: [A], [B], [C], [D], [E], [F], [G], [H]

1. NAIVE

- one thread processes everything

$$\begin{array}{l} A \\ A+B \\ A+B+C \\ \dots \end{array}$$

[A], [B], [C], [D], [E], [F], [G], [H]

↓ Thread 0
↓ output

threads: 1
shmem: no
syncthreads: no
parallel: no
reduction: no

2. Interleaved shmem

index: 0 1 2 3 4 5 6 7
value: [A], [B], [C], [D], [E], [F], [G], [H]

R1 → stride = 1

T0: 0 and 1) T0 T2: 4 and 5) T4
T1: 2 and 3) T2 T3: 6 and 7) T6

R2 → stride = 2

T0: 0 and 2) T0
T4: 4 and 6) T4

(active threads on each round)

R3 → stride = 4

T0: 0 and 4 ⇒ [A], [B], [C], [D], [E], [F], [G], [H]
T0

3. Sequential address

index: 0 1 2 3 4 5 6 7
value: [A], [B], [C], [D], [E], [F], [G], [H]

R1 $\rightarrow$ stride = 4

T0: A and E) T0

T2: C and G) T2

T1: B and F) T1

T3: D and H) T3

R2 $\rightarrow$ stride = 2

T0: 0 and 2) T0

T1: 1 and 3) T1

R3 $\rightarrow$ stride = 1

T0: 0 and 1) T0

[A], [B], [C], [D], [E], [F], [G], [H]
T0

! Interleaved: T0 -- T2 -- T4 -- T6 --
Sequential: T0 T1 T2 T3 -- -- --

4. Warp-shuffle

$\rightarrow$ mathematical tree looks same as sequential
the difference is where the values are stored

shmem: thread register
 $\downarrow$
shared[tid]

warp: thread register
 $\downarrow$
another lane reads
that register directly

↓
another thread
reads shared[]



Naive

↓ introduce parallel threads
Interleaved shared memory
↓ organize active threads better
Sequential addressing
↓ avoid most shared-memory communication
Warp shuffle

the Term AtomicAdd():

before `out[blockIdx.x] = shared[0];`

↳ `out[256, 256, 256, 232]`

↳ this requires another reduction second kernel launch!

with atomic add `atomicAdd(out, shared[0])`

↳ out starts at 0

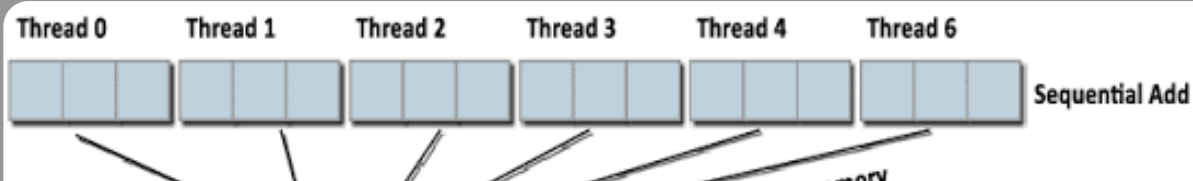
+ 256

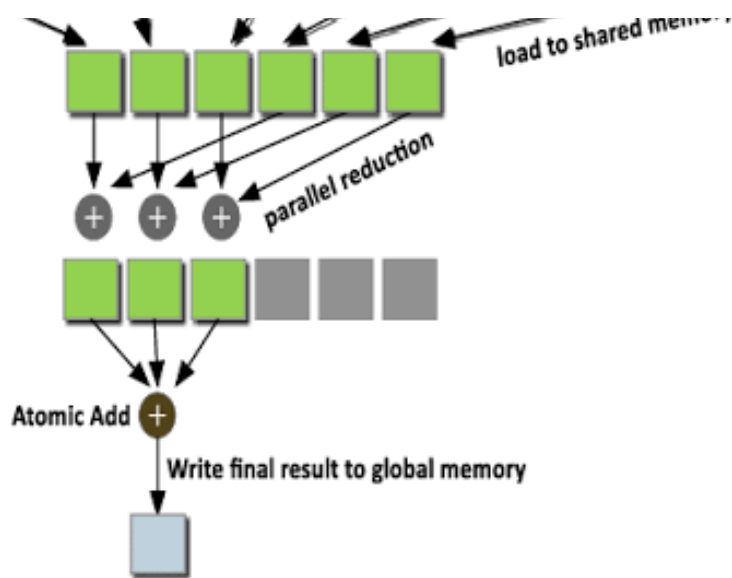
+ 256

+ 256

+ 232 = 1000

- Shared-memory reduction still combines threads inside each block.
- `atomicAdd` combines results between blocks.
- Output changes from an array of partial sums to one final scalar.
- `cudaMemset` is necessary because atomic addition must start from zero.
- The tradeoff is that atomic updates to the same address happen one at a time, so many blocks can create contention. For this simple task, one atomic per block is reasonable.





"why can one thread not efficiently compute the entire reduction?"

⇒ one thread processes everything instead of spreading across multiple workers

"why is synchronization required?"

⇒ make sure threads can see the data from other threads; to prevent data races

(?) "when can threads communicate w/out `_syncthreads()`?"

⇒ threads within the same warp can exchange register values using warp-shuffle instructions

"why is the warp-shuffle version often faster?"

⇒ warp-shuffle requires less shmem and fewer block-wide sync.

"why might GPU and CPU sums differ slightly?"

⇒ floating-point math is not associative